

Exemplo de BNF

- **Legenda para o uso da BNF:** variáveis (símbolos não-terminais) estão entre < e >; símbolos terminais estão em negrito; ::= é o simbolismo usado para geração, onde em LFA usávamos -> ; o conteúdo entre [e] é um conteúdo que pode existir ou não, se existir, será apenas uma vez; o conteúdo que está entre { e } é um conteúdo que pode ser gerado nenhuma ou inúmeras vezes, em LFA usávamos o símbolo *.
- Exemplo de programa aceito pela BNF descrita neste arquivo:

```
programa MODELO;  
var x, y, z: inteiro;  
    logica: booleano;  
  
procedimento soma;  
inicio  
    z := +x+y;  
fim  
  
inicio  
    escreva(z);  
fim
```

- Exemplo de programa **não** aceito pela BNF descrita neste arquivo:

```
programa MODELO-ERRADO;  
var x, y, z: inteiro;  
    logica: booleano;  
  
funcao subtracao: inteiro;  
inicio  
    z := x-y;  
    retorna z;  
fim  
  
inicio  
    escreva(subtracao);  
fim
```

Erros: linha 1: Não há "-" no identificador;

linha 7: não existe a palavra chave "retorna" na BNF; nessa BNF a função não retorna nada.

Descrição BNF da Linguagem Simplificada (BNF SUJEITA A MODIFICAÇÃO)

<programa> ::= **programa** <identificador> ; <bloco> .

<bloco> ::= [<etapa de declaração de variáveis>]
 [<etapa de declaração de sub-rotinas>]
 <comandos>

DECLARAÇÕES

<etapa de declaração de variáveis> ::= **var** <declaração de variáveis> ;
 {<declaração de variáveis>;}

<declaração de variáveis> ::= <identificador> {, <identificador>} : <tipo>

<tipo> ::= (**inteiro** | **booleano**)

<etapa de declaração de sub-rotinas> ::= (<declaração de procedimento>;|
 <declaração de função>;)
 {<declaração de procedimento>;|
 <declaração de função>;}

<declaração de procedimento> ::= **procedimento** <identificador>;
 <bloco>

<declaração de função> ::= **funcao** <identificador>: <tipo>;
 <bloco>

COMANDOS

<comandos> ::= **inicio**
 <comando>{;<comando>}[;]
 fim

<comando> ::= (<atribuição_chprocedimento>|
 <comando condicional> |
 <comando enquanto> |
 <comando leitura> |
 <comando escrita> |
 <comandos>)

<atribuição_chprocedimento> ::= (<comando atribuicao>|
 <chamada de procedimento>)

<comando atribuicao> ::= <identificador> := <expressão>

<chamada de procedimento> ::= <identificador>

<comando condicional> ::= **se** <expressão>
 entao <comando>
 [**senao** <comando>]

<comando enquanto> ::= **enquanto** <expressão> **faca** <comando>

<comando leitura> ::= **leia** (<identificador>)

<comando escrita> ::= **escreva** (<identificador>)

EXPRESSÕES

<expressão> ::= <expressão simples> [<operador relacional><expressão simples>]

<operador relacional> ::= (< > | = | < | <= | > | >=)

<expressão simples> ::= [+ | -] <termo> {(+ | - | **ou**) <termo> }

<termo> ::= <fator> {(* | **div** | **e**) <fator> }

<fator> ::= (<variável> |
 <número> |
 <chamada de função> |
 (<expressão>) | **verdadeiro** | **falso**
 nao <fator>)

<variável> ::= <identificador>

<chamada de função> ::= <identificador> >

NÚMEROS E IDENTIFICADORES

<identificador> ::= <letra> {<letra> | <dígito> | _ }

<número> ::= <dígito> {<dígito>}

<dígito> ::= (0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9)

<letra> ::= (a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z|
A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z)

COMENTÁRIOS

Uma vez que os comentários servem apenas como documentação do código fonte, ao realizar a compilação deste código faz-se necessário eliminar todo o conteúdo entre seus delimitadores.

delimitadores : { }